

Intermediate (build the gate) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

All examples below assume `versions.json` already has `v1` (score 0.6) and `v2` (score 1.0) saved, the same way the `version_log` exercise builds it, and reuse the `run_eval_suite` stub from that exercise.

Basic Version

Approach 1 — the direct way

```
import json

def load_versions(path):
    with open(path) as f:
        return json.load(f)

def run_eval_suite(prompt_text):
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def deploy(version_name, threshold):
    versions = load_versions("versions.json")
    target = None
    for v in versions:
        if v["version_name"] == version_name:
            target = v

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        print(version_name, "deployed, score", result["score"])
    else:
        print(version_name, "NOT deployed, score", result["score"], "below thresh")
```

```
deploy("v2", 0.9)
deploy("v1", 0.9)
```

Expected output:

```
v2 deployed, score 1.0
v1 NOT deployed, score 0.6 below threshold 0.9
```

And `live_version.txt` on disk after both calls contains just `v2` — the rejected `v1` call never touched it. This version works correctly for what the exercise asks. It crashes with a confusing `AttributeError` if `version_name` doesn't exist in the log at all (`target` stays `None`) — that's what Advanced fixes.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — a not-found guard, and a way to read the pointer back

```
import json

def load_versions(path: str) -> list[dict]:
    with open(path) as f:
        return json.load(f)

def find_version(versions: list[dict], name: str) -> dict | None:
    for v in versions:
        if v["version_name"] == name:
            return v
    return None

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def deploy(version_name: str, threshold: float) -> None:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        print(f"No such version: {version_name}")
        return

    result = run_eval_suite(target["prompt_text"])
    if result["score"] >= threshold:
        with open("live_version.txt", "w") as f:
            f.write(version_name)
        print(f"{version_name} deployed – score {result['score']} >= {threshold}")
    else:
        print(f"{version_name} NOT deployed – score {result['score']} < {threshold}")

def get_live_version() -> str | None:
    try:
        with open("live_version.txt") as f:
            return f.read().strip()
    except FileNotFoundError:
```

```
        return None

    deploy("v2", 0.9)
    deploy("v1", 0.9)
    deploy("v9", 0.9)
    print("currently live:", get_live_version())
```

Expected output:

```
v2 deployed – score 1.0 >= 0.9
v1 NOT deployed – score 0.6 < 0.9
No such version: v9
currently live: v2
```

Difference from Basic: Approach 1 splits the lookup into its own `find_version` helper with a real not-found path (`if target is None: ... return`) instead of letting a missing version crash on `target["prompt_text"]` , adds full type hints, and adds `get_live_version()` so the pointer can be read back safely (handling the case where `live_version.txt` doesn't exist yet). The score comparison and file-write logic are otherwise the same as Basic.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a structured `DeployResult`, and a raised not-found error

```
from dataclasses import dataclass, field

class VersionNotFoundError(Exception):
    pass

@dataclass
class DeployResult:
    passed: bool
    score: float
    failing_cases: list = field(default_factory=list)

def deploy(version_name: str, threshold: float) -> DeployResult:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(target["prompt_text"])
    passed = result["score"] >= threshold

    if passed:
        with open("live_version.txt", "w") as f:
            f.write(version_name)

    return DeployResult(
        passed=passed,
        score=result["score"],
        failing_cases=result.get("failed", []),
    )

outcome = deploy("v2", 0.9)
print(outcome)

outcome = deploy("v1", 0.9)
print(outcome)

try:
```

```

    deploy("v9", 0.9)
except VersionNotFoundError as e:
    print(f"Deploy failed: {e}")

```

Expected output:

```

DeployResult(passed=True, score=1.0, failing_cases=[])
DeployResult(passed=False, score=0.6, failing_cases=[])
Deploy failed: No saved version named: v9

```

`outcome.passed` and `outcome.score` are now real fields a caller can check directly — this is exactly what the Build Task's `deploy_gate.py` builds on, instead of parsing printed text.

Approach 2 — same idea, plain-dict `DeployResult` (no `dataclasses` import)

```

class VersionNotFoundError(Exception):
    pass

def deploy(version_name: str, threshold: float) -> dict:
    versions = load_versions("versions.json")
    target = find_version(versions, version_name)
    if target is None:
        raise VersionNotFoundError(f"No saved version named: {version_name}")

    result = run_eval_suite(target["prompt_text"])
    passed = result["score"] >= threshold

    if passed:
        with open("live_version.txt", "w") as f:
            f.write(version_name)

    return {
        "passed": passed,
        "score": result["score"],
        "failing_cases": result.get("failed", []),
    }

outcome = deploy("v2", 0.9)
print(outcome["passed"], outcome["score"])

```

Expected output:

```

True 1.0

```

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate prints the outcome and lets the not-found case fall through as a quiet early `return` — fine for a human watching the terminal, awkward for other code to build on. Both Advanced approaches replace that quiet `return` with a raised `VersionNotFoundError` (so a caller can't accidentally treat a missing version as "deployed: no"), and both return the score and pass/fail as real data instead of only printed text. Approach 1's `@dataclass` gives you `.passed` / `.score` attribute access and a readable `repr()` for free; Approach 2's plain dict needs no extra import and is just as usable, at the cost of `outcome["passed"]` instead of `outcome.passed`.

Which one should you actually write? Approach 1 (the dataclass) — it's what the Build Task's `deploy_gate.py` should return from its own `deploy()`, since `DeployResult` is named directly in the Build Task's requirements. Approach 2 is worth knowing as the "no extra import" fallback, but a typed `DeployResult` with real field names is easier for anyone reading `deploy_gate.py` later to trust at a glance, which matters most for exactly the kind of function — a release gate — that people need to trust without re-reading its internals every time.